

Kompendium Programowania w Java

Wersja Rozszerzona o Konkretne Przykłady - Klasa `java.lang.Math` oraz I/O (Pliki)

Zaktualizowana wersja dokumentu zawiera dodatkowe, małe szablony kodów i bezpośrednie przykłady pokazujące, **jak w praktyce rozwiązywać najczęstsze zadania kolokwialne** przy użyciu biblioteki `Math` oraz wejścia/wyjścia (I/O).

1. Klasa `java.lang.Math` — Matematyka w Java

Klasa `Math` zawiera zestaw statycznych metod i stałych służących do wykonywania podstawowych operacji numerycznych. **Nie wymaga importowania.**

1.1. Przegląd Najważniejszych Metod (Tabela Skrócona)

Sygnatura Metody	Zwracany Typ	Krótki Opis
<code>Math.abs(x)</code>	(Zgodny z arg)	Wartość bezwzględna.
<code>Math.max(a, b)</code>	(Zgodny z arg)	Zwraca większą z dwóch wartości.
<code>Math.pow(base, exp)</code>	<code>double</code>	Potęgowanie. Zawsze zwraca double!
<code>Math.sqrt(x)</code>	<code>double</code>	Pierwiastek kwadratowy.
<code>Math.round(x)</code>	<code>int</code> / <code>long</code>	Matematyczne zaokrąglenie do najbliższej całkowitej.

1.2. PRAKTYCZNE PRZYKŁADY: Jak tego używać na kolokwium?

Przykład A: Zaokrąglenie liczby do 2 miejsc po przecinku (Pewniak!)

Klasa `Math` zaokrągla tylko do liczb całkowitych. Jeśli każą Ci zaokrąglić wynik do 2 miejsc po przecinku (np. kwoty), użyj tego triku:

```
double wynik = 15.456789;
// 1. Mnożymy przez 100 -> 1545.6789
// 2. Math.round zaokrągla -> 1546
// 3. Dzielimy przez 100.0 (z kropką, żeby to był double!) -> 15.46
double zaokrąglony = Math.round(wynik * 100.0) / 100.0;
System.out.println(zaokrąglony); // Wypisze: 15.46
```

Przykład B: Twierdzenie Pitagorasa (Obliczanie długości/odległości)

Gdy trzeba policzyć odległość między dwoma punktami albo długość wektora, łączymy potęgowanie i pierwiastek:

```
double a = 3.0;
double b = 4.0;
// Wzór c = sqrt(a^2 + b^2)
double c = Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2));
System.out.println("Przeciwprostokątna: " + c); // Wypisze: 5.0
```

Przykład C: Wybieranie największej z trzech lub więcej liczb

Metoda `Math.max()` przyjmuje tylko dwa argumenty. Co, jeśli masz 3 zmienne (x, y, z)? Trzeba je "zagnieździć":

```
int x = 10, y = 25, z = 15;
// Najpierw znajdujemy większą z (x, y), a potem wynik porównujemy z z
int najwieksza = Math.max(Math.max(x, y), z);
System.out.println("Największa to: " + najwieksza); // Wypisze: 25
```

Przykład D: Generowanie Liczb Losowych z Przedziału [MIN, MAX]

```
int min = 10;
int max = 50;
// Wzór: (int)(Math.random() * (max - min + 1)) + min
int losowaLiczba = (int)(Math.random() * (50 - 10 + 1)) + 10;
```

2. Zapisywanie i Odczyt Danych - Rozwiązania Zadań

Poniżej znajdują się szybkie "wzorce projektowe" do rozwiązania 90% zadań kolokwialnych z operacji na plikach.

2.1. Zapis Tablicy / Listy do Pliku z użyciem PrintWriter

Założmy zadanie: "Wygeneruj tablicę 10 losowych liczb i zapisz je do pliku liczby.txt, każda w nowej linii."

```
import java.io.*;

public class ZapisTablicy {
    public static void main(String[] args) {
        int[] tablica = { 14, 52, 3, 88, 21 };

        try (PrintWriter pw = new PrintWriter(new FileWriter("liczby.txt"))) {
            for (int liczba : tablica) {
                pw.println(liczba); // println zapewnia znak nowej linii
            }
            System.out.println("Zapisano poprawnie!");
        } catch (IOException e) {
            System.err.println("Błąd zapisu: " + e.getMessage());
        }
    }
}
```

2.2. Odczyt Pliku: Zliczanie Sumy i Średniej (Klasa Scanner)

Zadanie: "W pliku dane.txt w każdej linii jest jedna liczba. Wczytaj je, oblicz ich sumę i średnią."

```
import java.io.File;
import java.util.Scanner;

public class SumaZeSkanera {
    public static void main(String[] args) {
        double suma = 0;
        int ileLiczby = 0;

        try (Scanner skaner = new Scanner(new File("liczby.txt"))) {
            // Zamiast nextLine() korzystamy z hasNextDouble(), co ułatwia sprawę!
            while (skaner.hasNextDouble()) {
                double wczytanaLiczba = skaner.nextDouble();
                suma += wczytanaLiczba;
                ileLiczby++;
            }

            if (ileLiczby > 0) {
                System.out.println("Suma: " + suma);
                System.out.println("Średnia: " + (suma / ileLiczby));
            }
        } catch (Exception e) {
            System.err.println("Nie można przetworzyć pliku!");
        }
    }
}
```

2.3. Zaawansowany Odczyt: Dzielenie wierszy w pliku CSV

Zadanie: "Wczytaj plik, gdzie dane są rozdzielone średnikiem (np. 'Adam;Kowalski;25'). Wyciągnij sam wiek i sprawdź czy osoba jest pełnoletnia."

```
import java.io.File;
import java.util.Scanner;

public class ParsowanieLinii {
    public static void main(String[] args) {
        try (Scanner skaner = new Scanner(new File("osoby.csv"))) {
            while (skaner.hasNextLine()) {
                String linia = skaner.nextLine();

                // Metoda split() rozcina linijkę (String) na tablicę pod-Stringów
                String[] czesci = linia.split(";");

                // Mamy tablicę np. ["Adam", "Kowalski", "25"]
                String imie = czesci[0];
                int wiek = Integer.parseInt(czesci[2]); // Zmiana napisu "25" na liczbę 25

                if (wiek >= 18) {
                    System.out.println(imie + " jest pełnoletni(a).");
                }
            }
        } catch (Exception e) {
            System.err.println("Błąd: " + e.getMessage());
        }
    }
}
```

Pamiętaj! Metoda `split(",")` tworzy tablicę z fragmentów. Zawsze używaj klas osłonowych takich jak `Integer.parseInt(tekst)` lub `Double.parseDouble(tekst)`, by zmienić wyciągnięty tekst w faktyczną zmienną numeryczną, na której można robić obliczenia.